

Lesson 09: Terraform Data Sources

Learning Objectives

- Understand what data sources are and when to use them
- Read existing AWS resources using data sources
- Combine data sources with resources for cross-reference patterns
- Use data source filtering and querying

1. What are Data Sources?

A **data source** queries an existing resource that was **created outside of Terraform** (or in a different Terraform configuration) and makes its attributes available for use.

```
# Query an existing bucket (not managed by this Terraform config)
data "aws_s3_bucket" "existing" {
  bucket = "bucket-created-manually"
}
```

Resources vs Data Sources

	Resource	Data Source
Creates something?	Yes	No (read-only)
Manages lifecycle?	Yes (create, update, delete)	No
Syntax	resource "type" "name"	data "type" "name"
Reference	resource_type.name.attribute	data.data_type.name.attribute
Use case	Provision new infrastructure	Read existing infrastructure

2. Data Source Syntax

```
data "aws_s3_bucket" "existing" {
  bucket = "my-existing-bucket"
}
```

Arguments are used to **filter/identify** the resource. After `terraform apply` (or `terraform plan` for data sources), Terraform fetches the attributes.

Using Data Source Attributes

```
# Reference data sources with: data.<type>.<name>.<attribute>
output "bucket_arn" {
  value = data.aws_s3_bucket.existing.arn
}

output "bucket_region" {
  value = data.aws_s3_bucket.existing.region
}
```

3. Common Data Sources

S3 Bucket

```
# Query an existing S3 bucket
data "aws_s3_bucket" "existing" {
  bucket = "bucket-created-outside-terraform"
}

# Use its attributes
output "arn"          { value = data.aws_s3_bucket.existing.arn }
output "id"           { value = data.aws_s3_bucket.existing.id }
output "bucket_region" { value = data.aws_s3_bucket.existing.bucket_region
}
```

DynamoDB Table

```
data "aws_dynamodb_table" "existing" {
  name = "ExistingTable"
}

output "table_arn"          { value = data.aws_dynamodb_table.existing.arn }
output "table_id"           { value = data.aws_dynamodb_table.existing.id }
output "table_name"         { value =
data.aws_dynamodb_table.existing.name }
```

SQS Queue

```
data "aws_sqs_queue" "existing" {
  name = "existing-queue"
}
```

```
output "queue_arn" { value = data.aws_sqs_queue.existing.arn }
output "queue_url" { value = data.aws_sqs_queue.existing.url }
```

IAM Policy Document (data-only, no existing resource needed)

```
data "aws_iam_policy_document" "bucket_policy" {
  statement {
    sid      = "AllowSQSWrite"
    effect   = "Allow"

    principals {
      type        = "Service"
      identifiers = ["sqs.amazonaws.com"]
    }

    actions   = ["s3:PutObject"]
    resources = ["arn:aws:s3:::my-bucket/*"]
  }
}
```

AWS Caller Identity (current account/region)

```
data "aws_caller_identity" "current" {}

output "account_id" { value = data.aws_caller_identity.current.account_id }
output "arn"        { value = data.aws_caller_identity.current.arn }
output "user_id"    { value = data.aws_caller_identity.current.user_id }
```

AWS Region

```
data "aws_region" "current" {}

output "region_name" { value = data.aws_region.current.region }
output "region_description" { value = data.aws_region.current.description }
```

4. Cross-Reference: Resource + Data Source

A common pattern: create some resources with Terraform, then use data sources to read them back for cross-referencing.

```
# Create a bucket
resource "aws_s3_bucket" "created" {
  bucket = "tf-created-bucket"
```

```
tags = {
  Environment = "dev"
}

# Read the same bucket back via data source
data "aws_s3_bucket" "created" {
  bucket = aws_s3_bucket.created.id
}

# Use both resource and data source attributes
output "resource_arn" { value = aws_s3_bucket.created.arn }
output "data_source_arn" { value = data.aws_s3_bucket.created.arn }
```

Both will show the same ARN. The data source can access attributes that the resource might not expose directly.

5. Combining Data Sources with Resources

```
# data.tf
data "aws_s3_bucket" "existing" {
  bucket = var.existing_bucket_name
}

data "aws_dynamodb_table" "existing" {
  name = var.existing_table_name
}

# main.tf – use data source attributes in new resources
resource "aws_sqs_queue" "notifications" {
  name = "notifications-queue"

  tags = {
    SourceBucket = data.aws_s3_bucket.existing.id
    SourceTable  = data.aws_dynamodb_table.existing.arn
  }
}

# outputs.tf
output "existing_bucket_arn" { value = data.aws_s3_bucket.existing.arn }
output "existing_table_arn"  { value = data.aws_dynamodb_table.existing.arn }
output "new_queue_arn"      { value = aws_sqs_queue.notifications.arn }
```

6. Reading Resources Created by AWS CLI/UI

This workflow demonstrates the power of data sources. Create a resource manually via CLI, then reference it in Terraform:

Step 1: Create manually

```
aws --endpoint-url=http://localhost:4566 s3 mb s3://manually-created-bucket
aws --endpoint-url=http://localhost:4566 s3api put-bucket-tagging \
  --bucket manually-created-bucket \
  --tagging 'TagSet=[{Key=Environment,Value=manual},
{Key=Source,Value=CLI}]'
```

Step 2: Reference in Terraform

```
data "aws_s3_bucket" "manual" {
  bucket = "manually-created-bucket"
}

output "manual_bucket_arn" {
  value = data.aws_s3_bucket.manual.arn
}
```

Step 3: Run

```
terraform apply
```

Terraform reads the existing bucket's attributes and displays them — without creating, modifying, or deleting it.

7. The `aws_iam_policy_document` Data Source

This is a special data source that generates a JSON IAM policy document:

```
data "aws_iam_policy_document" "s3_policy" {
  statement {
    sid      = "AllowPublicRead"
    effect   = "Allow"

    principals {
      type        = "*"
      identifiers = ["*"]
    }

    actions   = ["s3:GetObject"]
    resources = ["arn:aws:s3:::public-bucket/*"]
  }
}
```

```
statement {
  sid      = "DenyInsecureConnections"
  effect   = "Deny"

  principals {
    type       = "*"
    identifiers = ["*"]
  }

  actions   = ["s3:*"]
  resources = ["arn:aws:s3:::public-bucket/*"]

  condition {
    test       = "Bool"
    variable   = "aws:SecureTransport"
    values     = ["false"]
  }
}

output "policy_json" {
  value = data.aws_iam_policy_document.s3_policy.json
}
```

8. Complete Example

```
# variables.tf
variable "existing_bucket" {
  description = "Name of an existing S3 bucket"
  type        = string
}

variable "existing_table" {
  description = "Name of an existing DynamoDB table"
  type        = string
}

# data.tf
data "aws_s3_bucket" "existing" {
  bucket = var.existing_bucket
}

data "aws_dynamodb_table" "existing" {
  name = var.existing_table
}

data "aws_caller_identity" "current" {}

data "aws_region" "current" {}
```

```
# main.tf
resource "aws_sqs_queue" "notification" {
  name = "notification-queue"

  tags = {
    SourceBucket = data.aws_s3_bucket.existing.id
    AccountID     = data.aws_caller_identity.current.account_id
    Region        = data.aws_region.current.region
  }
}

# outputs.tf
output "existing_bucket_arn" { value = data.aws_s3_bucket.existing.arn }
output "existing_table_arn" { value =
data.aws_dynamodb_table.existing.arn }
output "existing_table_name" { value =
data.aws_dynamodb_table.existing.name }
output "account_id"         { value =
data.aws_caller_identity.current.account_id }
output "region"             { value = data.aws_region.current.region }
output "new_queue_arn"      { value = aws_sqs_queue.notification.arn }
```

9. Key Takeaways

- Data sources are **read-only** — they query existing resources without managing them
- Reference data source attributes as `data.<type>.<name>.<attribute>`
- Data sources work with resources created by CLI, Web UI, or other Terraform configs
- Filter arguments (like `bucket`, `name`) identify which resource to read
- Common built-in data sources: `aws_caller_identity`, `aws_region`, `aws_iam_policy_document`
- Data sources are resolved during `terraform plan/apply` — they don't need a separate command

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer – Eyuel M.